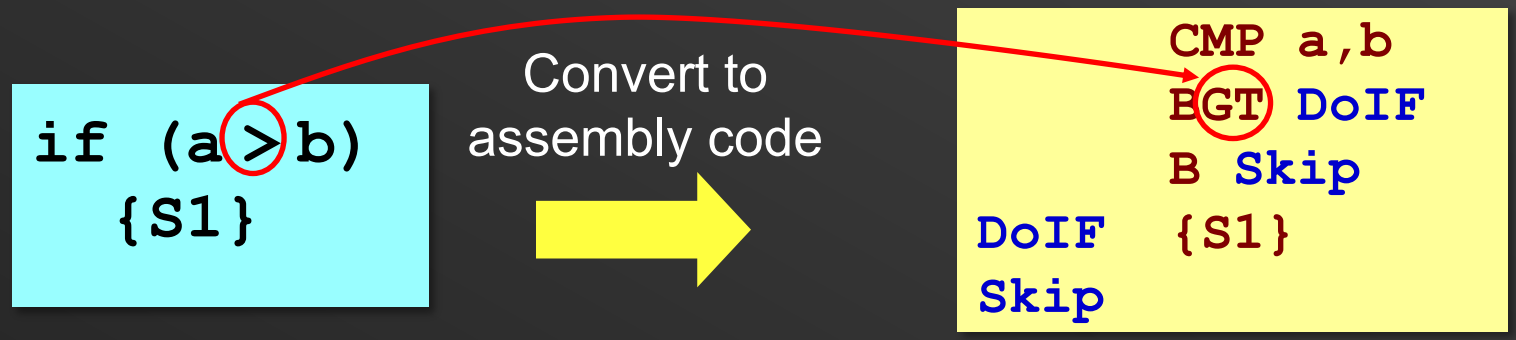
Chapter 7: Flow-control Constructs (live lecture)

Mohamed M. Sabry Aly

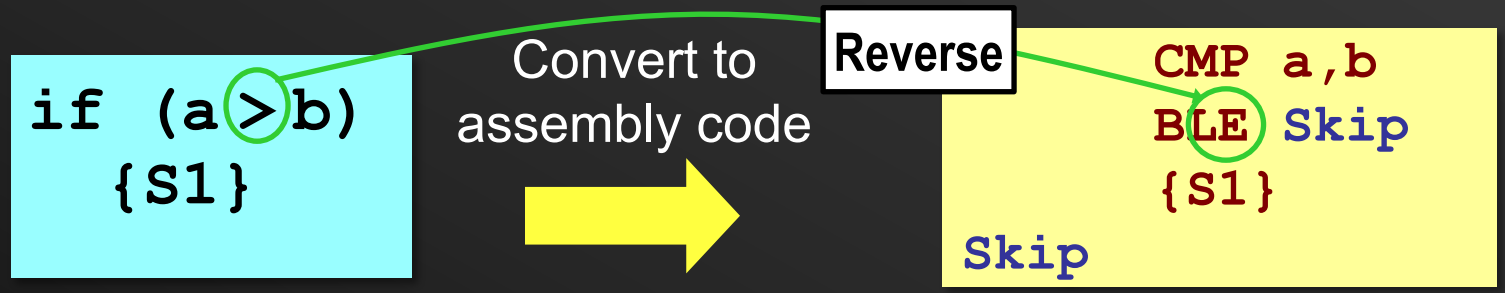
N4-02c-92

IF Statements

- How is the **IF** construct implemented?
- **Imitating** the high-level test condition does not result in very efficient assembly-level implementation.



- High-level test condition is **reversed** in assembly-level to avoid the need for an additional unconditional jump.



Note: The reverse condition test of **HS** is **LO**, reverse of **LT** is **GE**

Review Q1 - IF

What is the equivalent C code segment for the following ARM code.

```
        CMP R2,R3
        BLT Next
        ADD R3,R3,R2
Next    :
```

(1)

```
if (R2 < R3)
  R3 = R3 + R2;
```

(2)

```
if (R2 < R3)
  R2 = R2 + R3;
```

(3)

```
if (R2 >= R3)
  R3 = R2 + R3;
```

(4)

```
if (R3 >= R2)
  R3 = R2 + R3;
```

3 is the correct answer

Conditional Execution

- In the 32-bit ARM ISA, instructions can be conditionally executed based on the CC flags.
- Consider the following 32-bit ARM code segment.

```
; C code
if (r0 == 1)
 r1 = 3;
```

```
        CMP   r0, #1        ; set CC based on r0 –1
        BNE   Skip          ; if (r0 == 1)
        MOV   r1, #3        ; then { r1 := 3}
SKIP      ......            ;
```

- It can be replaced using conditional execution instructions.

```
        CMP    r0, #1       ; if (r0 == 1)
        MOVEQ  r1, #3       ; then { r1 := 3}
SKIP    ......              ;
```

Find Largest Number, and Store Its Index

```
R3= X[0];
R4=0;
R0= &X[0]
For (R1=9;R1>0;R1--){
        R0+=4;R2=X[R0];
       if(R2>=R3){
              R3=R2;
              R4=10-R1;
       }
}
```

No conditional execution

```
        MOV   R0,#0x100      ;setup pointer to first array element
        MOV   R1,#9          ;load 9 into counter register
        MOV   R4,#0          ;load 0 into index_max register
        LDR   R3,[R0]        ; 1st no. in array is current max
Loop    LDR   R2,[R0,#4]!    ;get next no. in array
        CMP   R2,R3          ;compare R3 and R2 (i.e. R2-R3)
        BLO   Skip           ;branch if R2 <  current max (i.e.
        MOV   R3,R2          ;update current max. in R3 with R2
        RSUB  R4,R1,#10      ;Store the index in R4
Skip    SUBS  R1,R1,#1       ;decrement 1 from counter register
        BNE   Loop           ;jump back to Loop if not zero
```

Find Largest Number, and Store Its Index

```
R3= X[0];
R4=0;
R0= &X[0]
For (R1=9;R1>0;R1--){
        R0+=4;R2=X[R0];
       if(R2>=R3){
              R3=R2;
              R4=10-R1;
       }
}
```

With conditional execution

```
        MOV   R0,#0x100      ;setup pointer to first array element
        MOV   R1,#9          ;load 9 into counter register
        MOV   R4,#0          ;load 0 into index_max register
        LDR   R3,[R0]        ; 1st no. in array is current max
Loop    LDR     R2,[R0,#4]!  ;get next no. in array
        CMP     R2,R3        ;compare R3 and R2 (i.e. R2-R3)
        MOVGE   R3,R2        ;update current max. in R3 with R2
        RSUBGE  R4,R1,#10    ;Store the index in R4
        SUBS    R1,R1,#1     ;decrement 1 from counter register
        BNE   Loop           ;jump back to Loop if not zero
```

Conditional Execution for IF-ELSE

- Higher efficient coding
- In the shown example, Skip will always be the 4th instruction

```
; C code
if (r0 == 1)
 r1 = 3;
else
 r1 = 5;
```

```
      CMP   r0, #1        ; set CC based on r0 –1
      BNE   ELSE          ; if (r0 == 1)
      MOV   r1, #3        ; then { r1 := 3}
      B     SKIP          ; skip over else code seg
ELSE  MOV   r1, #5        ; else  { r1 := 5}
SKIP    ......            ;
```

- With conditional execution

```
      CMP  r0, #1         ; if (r0 == 1)
      MOVEQ r1, #3        ; then { r1 := 3}
      MOVNE r1, #5        ; else  { r1 := 5}
SKIP   ......             ;
```

What is the most optimal implementation of this C-code

```
If (X>10)
      F1(X);
Else
      F2(X);
```

(1)

```
CMP    R0, #10
BLGT   F1
BL     F2
```

(2)

```
CMP          R0,#10
BGT          Doif
BL           F2
B            Done
Doif         BL F1
Done
```

(3)

```
CMP      R0, #10
BLLE     F2
CMP      R0,#10
BLGT     F1
```

(4)

```
CMP          R0,#10
BLT          DoElse
BL           F1
B            Done
Doelse       BL F2
Done
```

3 is the answer

Compound AND Conditions

- How are compound AND conditions handled?
- Logical AND can bind multiple basic relational conditions.

e.g.

```
if ((a==b) && (b>0)) {S1}
```

order of compound AND test

- Compilers resolve compound conditions into simpler ones.

```
    if (a!=b) then Skip
    if (b<=0) then Skip
       {S1}
Skip    :
```

- Elementary conditions bound by the logical AND are tested from **left-to-right**, in the order given in the C program.
- The first false condition means the remaining conditions are not computed. This is called the **fast Boolean operation**.
- Keep the **least likely** condition **leftmost** in your program for more efficient execution.

Compound Condition Example

- Not all cases will be executed correctly

```
if ((a==b) && (b>0)) {S1}
```

What if a>b and b>0

```
CMP     R1,R2  ;R1<-a, R2<-b
CMPEQ   R2,#0
XXXGT   XXXX   ; S1
```

Compound OR Conditions

- How are compound OR conditions handled?

e.g.

```
if ((a == 1) || (a == 2)) {S1}
```

- Most compilers eliminate an unconditional jump at the end of the compound OR series by **reversing** the **last conditional** test.

```
        if (a == 1) then DoIf
        if (a != 2) then Skip
DoIf    {S1}
Skip      :
```

- The conditional test that is **most likely** to be **true** should be kept leftmost.

Compound Condition Example

```
if ((a == 1) || (a == 2)) {S1}
```

With conditional assignment
S1 needs to be replicated.
Not suitable for multiple instructions

```
CMP     R1,#1  ;R1<-a
XXXEQ   XXXX   ;  S1
CMPNE   R1,#2
XXXEQ   XXXX   ;  S1
```

A more holistic approach

```
        CMP     R1,#1
        BEQ     doIF
        CMPNE   R1,#2
doIF    XXXEQ   XXXX   ;  S1
```

What would be the C equivalent for the shown assembly?

```
        CMP R2,R3
        BNE Next
        CMP R3,#2
        ADDGE R3,R3,R2
Next :
```

(1)

```
if (R2 == R3 && R3 > 2)
  R3 = R3 + R2;
```

(2)

```
if (R2 != R3 || R3 < 2)
  R3 = R3 + R2;
```

(3)

```
if (R2 == R3 && R3 ≥ 2 )
  R3 = R3 + R2;
```

(4)

```
if (R3 != R2 && R3 ≥ 2 )
  R3 = R3 + R2;
```

Answer is 3

Switch Statement

- How is the **SWITCH** construct implemented?
- The assembly code produced varies between compilers and depends on the nature and range of the case values.
- Two different SWITCH scenarios are examined:

```
switch(x) {
 case 0 : {S0};
          break;
 case 1 : {S1};
          break

 case 2 : {S2};
          break;
 case 3 : {S3};
}
```

Running values narrow range

```
switch(x) {
 case 1    : {S0};
             break;
 case 10   : {S1};
             break

 case 100  : {S2};
             break;
 case 1000 : {S3};
}
```

Random values wide range

Jump Table Example

```
switch(x)
{
case 0:
 G='F';
 break;

case 1:
 G='D';
 break;

case 2:
 G='C';
 break;

case 3:
 G='B';
}
```

Cascaded if-else

```
      LDR R1, [R2]
      CMP R1, #0
      BEQ C1
      CMP R1, #1
      BEQ C2
      CMP R1, #2
      BEQ C3
      MOV R0, #66 ;'B'
RES   STR R0, [R2,#4]
      .
      .
      .
C1    MOV R0, #70 ;'F'
      B RES
C2    MOV R0, #68 ;'D'
      B RES
C3    MOV R0, #67 ;'C'
      B RES
```

Jump Table

```
      LDR R1, [R2]
      ADR R3, TBL
      LDR PC, [R3,R1, LSL #2]
RES   STR R0, [R2,#4]
      .
      .
0xA00 C1   MOV R0, #70 ;'F'
      B RES
C2    MOV R0, #68 ;'D'
      B RES
C3    MOV R0, #67 ;'C'
      B RES
```

Address	Contents
TBL + 0	0xA00
TBL + 4	0xA08
TBL + 8	0xA10
:	

Jump Table

Jump Table Example (Conditional Execution)

```
switch(x)
{
case 0:
 G='F';
 break;

case 1:
 G='D';
 break;

case 2:
 G='C';
 break;

case 3:
 G='B';
}
```

Cascaded if-else

```
      LDR R1, [R2]
      CMP R1, #0
      MOVEQ R0,#70
      CMP R1, #1
      MOVEQ R0,#68
      CMP R1, #2
      MOVEQ R0,#67
      CMP R1, #3
      MOVEQ R0, #66 ;'B'
RES   STR R0, [R2,#4]
      .
      .
      .
```

Jump Table

```
      LDR R1, [R2]
      ADR R3, TBL
      LDR PC, [R3,R1, LSL #2]
RES   STR R0, [R2,#4]
      .
      .
0xA00 C1   MOV R0, #70 ;'F'
      B RES
C2    MOV R0, #68 ;'D'
      B RES
C3    MOV R0, #67 ;'C'
      B RES
```

Address	Contents
TBL + 0	0xA00
TBL + 4	0xA08
TBL + 8	0xA10
:	

Jump Table

WHILE Implementation

- Implementation of the **WHILE** loop constructs:
- This is an example of a **pre-test** loop.
- If the condition **(VarX > 0)** is false, the loop segment is not executed at all.

```
WHILE (VarX > 0)
{
    Loop segment
}
```

```
Back   CMP "VarX",#0
       BLE Exit
        :
        :   } Loop segment
        :
       B Back
Exit    :
```

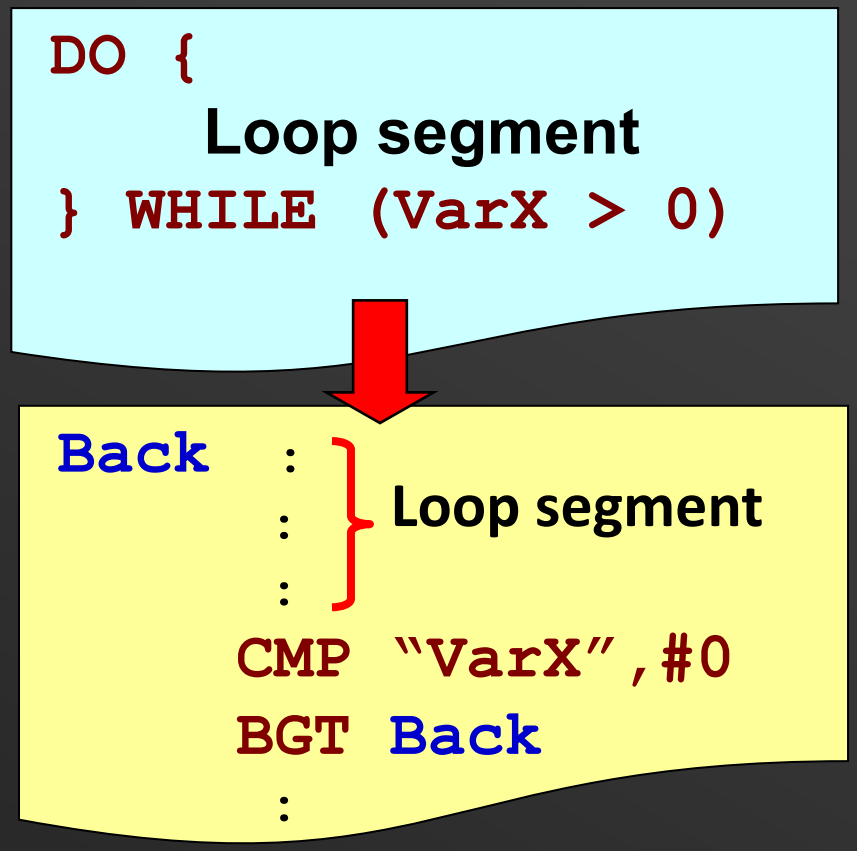
Implementation of WHILE in ARM assembly language

Note: **VarX** is variable. you will need to load it to a register.

DO-WHILE Implementation

- Implementation of the **DO-WHILE** loop constructs
- This is an example of a **post-test** loop.
- The loop segment is executed at least once before condition is tested.
- Post-test loop construct is **more efficient** than the pre-test as there is no need for an additional unconditional jump.

Note: `VarX` is variable. you will need to load it to a register.



Implementation of DO- WHILE in ARM assembly language

FOR Implementation

- Implementation of **FOR** loop constructs:

- The FOR loop is a **pre-test** loop that evaluates the condition first before executing loop segment.

- If loop segment is executed and count **N** is not used in loop segment, some optimizing compilers implement the **FOR** loop using a **post-test** with **decrement** & **test for zero**.

```
FOR (N=0; N<5; N++)
{
    Loop segment (x5)
}
```

```
        MOV R0,#0
Back    CMP R0,#5
        BGE Exit
         :
         :   } Loop segment
        ADD R0,R0,#1
        B   Back
Exit     :
```

Implementation of FOR in ARM assembly language

Review Q2 - Loop

What is the equivalent C code segment for the following ARM code.

```
Back  CMP  R0,#5
      BLT  Next
      SUB  R0,R0,#1
      B  Back
Next  :
```

(1)

```
FOR (X=5; X>0; X--) {
   }
```

(2)

```
WHILE (X >= 5) {
   X = X - 1;
   }
```

(3)

```
WHILE (X < 5) {
   X = X - 1;
   }
```

(4)

```
DO {
   X = X -1;
   } WHILE (X >= 5)
```

Answer is 2